作业 1：线性模型和支持向量机

清华大学软件学院
机器学习，2026 年春季学期

1 介绍

本次作业需要提交说明文档（PDF 形式）和 Python 的源代码。注意事项如下：

- 本次作业满分 105，若总得分超过 100 分，则按照 100 分截断。
- 作业按点给分，因此请在说明文档中按点回答，方便助教批改。示例如下：
  2.2.1 $J(\theta) = \frac{1}{m}\sum_{i=1}^{m}\left(h_\theta(x_i) - y_i\right)^2 + \cdots$。
- 本次作业除特殊说明外，自主实现的代码不能使用机器学习的开源库，如 sklearn、pytorch 等。
- 你需要熟练掌握 numpy 及其广播（broadcast）机制，**使用 for 循环实现的矩阵运算不会得到编程部分的分数**。
- 请通过提供易懂的注释或变量命名确保代码有足够的可读性，若代码的可读性过差，会酌情扣除对应题的分数。
- 在 PDF 中记录你在写程序的哪些模块时与他人有过交流，与他人交流需具体到姓名（网络论坛上的交流可填写用户名），参考网络资料需具体到链接。
- 如使用大模型辅助作业完成，请在 PDF 中声明作业中你使用了大模型的部分和使用的方式。**大模型仅可作为辅助，禁止直接粘贴大模型生成的回答，请确保充分掌握作业相关知识点**。
- 不要使用他人的作业（含代码和文档），也不要向他人公开自己的作业，否则处罚很严厉，会扣至-100（倒扣本次作业的全部分值）。

2 线性模型 (65pt)

在本题中，假设空间为线性函数，你将使用梯度下降法（Gradient Descent）拟合数据。你可以通过在 `start_code.py` 中搜索“TODO”标记来定位所有你需要完成的代码。你应该不需要修改其他部分代码，但你也可以根据需要做出合理调整。

2.1 数据处理：数据集划分和归一化（5pt）

在日后的科研过程中，同学们会看到很多开源数据集只有“训练集”和“测试集”。这是由于真正的测试集往往需要模型部署上线后才能获得，或者更抽象的说，真正的测试集是“the test of time”。故开源数据集中的“测试集”本质上是“验证集”。在本次作业中，也请同学们在初见机器学习时保证学术上的严谨，禁止使用测试集调参。同学们应选出验证集上表现最好的模型作为最终模型，并如实汇报此模型在测试集的表现，**测试集一经启用，不可继续调整模型**。只要同学们正确完成了模型的训练和验证集调参，测试集最终表现好坏不会影响给分。

在实际应用中，当数据的不同维特征差异很大时，梯度下降的收敛速度会变得很慢。此外，使用正则化时，具有较大绝对值的特征对正则化有更大的影响。因此，我们需要进行特征归一化。一种常见方法是执行仿射变换，将**训练集**中的所有特征值映射至 $[0,1]$，对验证集、测试集上的每个特征也需要使用相同的仿射变换。

1. 补全函数 `split_data`，将数据集划分为训练集、验证集和测试集。
2. 补全函数 `feature_normalization`，实现特征的归一化。

2.2 基函数与正则化（7pt）

现已知，代码中的 y_1 仅和 x_1, x_2, x_3 有关，即 $y_1 = f(x_1, x_2, x_3)$。

1. **基函数：** y_1 和 x_i 之间的依赖关系不完全是线性的。已知 y_1 和 $x_i, cos(x_i), x_i^2$ 之间的依赖关系是线性的。故需通过引入基函数，使用线性模型拟合 y_1。请补全函数 `build_basis_features` 完成基函数定义。
2. **一范数、二范数正则化：** 分别使用一范数和二范数正则化训练模型，并尝试不同的正则化强度。本题无需你补充代码，助教已使用 pytorch 库实现了梯度的计算和更新（在后续手动计算梯度的题目中，禁止直接使用 pytorch 功能）。你需要首先借助 pytorch 文档或大模型读懂代码，运行实验并简要回答以下两个问题：观察最终得到的模型的权重，尤其注意观察正则化权重比较大时的实验结果，联系课上相关知识简要解释你的实验现象；并直接写出你认为 $x_i, cos(x_i), x_i^2$，$i = 1, 2, 3$ 这九个自变量中，哪三个是比较重要的自变量。

2.3 目标函数与梯度（12pt）

现已知，代码中的 y_2 取值为连续值。在**线性回归**中，我们考虑线性函数的假设空间 $h_\theta : \mathbf{R}^d \to \mathbf{R}$,

$$h_{\theta,b}(x) = \theta^T x + b,$$

其中 $\theta, x \in \mathbf{R}^d$, 为了书写和编程的方便，我们通常为 x 添加一个额外的维度，该维度始终是一个固定值，例如 1，用于消除 h_θ 的偏移量参数 b。此时，我们可以将待优化的函数写成:

$$h_\theta(x) = \theta^T x,$$

其中 $\theta, x \in \mathbf{R}^{d+1}$。我们需要找到合适的 θ 最小化:

$$J_{\mathrm{MSE}}(\theta) = \frac{1}{m} \sum_{i=1}^{m} \left(h_\theta(x_i) - y_i\right)^2,$$

其中 $(x_1, y_1), \ldots, (x_m, y_m) \in \mathbf{R}^{d+1} \times \mathbf{R}$ 是训练数据。

岭回归是使用 L_2 正则化的线性回归，其目标函数是

$$J(\theta) = J_{\mathrm{MSE}}(\theta) + \lambda R(\theta) = \frac{1}{m} \sum_{i=1}^{m} (h_\theta(x_i) - y_i)^2 + \lambda \theta^T \theta,$$

其中 λ 是正则化参数，它控制正则化程度。

1. 将训练数据的特征记作 $X = (x_1, x_2, \ldots, x_m)^T \in \mathbf{R}^{m \times (d+1)}$ ，即 X 的第 i 行是 x_i^T, 训练数据的输出记作 $y = (y_1, y_2, \ldots, y_m)^T \in \mathbf{R}^m$，请写出 $J(\theta)$ 的矩阵形式。
2. 补全函数 `compute_regularized_square_loss`，给定 θ，计算的 $J(\theta)$。
3. 请写出 $J(\theta)$ 对 θ 梯度的矩阵形式。
4. 补全函数 `compute_regularized_square_loss_gradient`，给定 θ，计算 $J(\theta)$ 的梯度。

   为了检查梯度的实现是否正确，可以用数值法来计算梯度。可导函数的方向导数可以通过下述公式计算，

   $$\lim_{\varepsilon \to 0} \frac{J(\theta + \varepsilon h) - J(\theta - \varepsilon h)}{2\epsilon}$$

   在实际操作中，我们可以选择一个较小的 $\varepsilon > 0$，通过逼近方向导数来逼近梯度。更具体地，可以在每个坐标方向上首先取 $h = (1, 0, 0, \ldots, 0)^T$ 计算第一维的梯度，然后取 $h = (0, 1, 0, \ldots, 0)^T$ 得到第二维的梯度，以此类推。将每一维的梯度合起来，就得到 $J(\theta)$ 在 θ 处的梯度。我们已在代码中实现了函数 `grad_checker`，你可以用它检验梯度计算函数的正确性。

2.4 梯度下降 (6pt)

1. 在最小化 $J(\theta)$ 时，假设取 θ 到 $\theta + \eta h$ 的一步，其中 $h \in \mathbf{R}^{d+1}$ 是前进方向（不一定是单位向量），$\eta \in (0, \infty)$ 是步长。请用梯度写出目标函数值变化的近似表达式 $J(\theta + \eta h) - J(\theta)$，思考 h 为哪一前进方向时目标函数下降最快，并据此写出梯度下降中更新 θ 的表达式。
2. `main` 函数已经加载了数据，将其拆分成了训练集和测试集，并完成了归一化。你现在需要补全函数 `gradient_descent` 实现**梯度下降**算法，使得模型可以在训练集上进行训练。

2.5 模型选择 (5pt)

1. 首先补充 main 函数完成必要的训练和记录过程（使用不同的超参数调用 `grad_descent` 函数，记录实验结果）。我们可以通过**验证集**上的均方误差 $J_{\mathrm{MSE}}(\theta)$ 来选择合适的模型训练超参数（再次提醒不要在测试集调参）。用表格记录不同超参数下模型的验证集均方误差，搜索出最优的模型训练超参数（步长、正则化系数），汇报选出的最优超参数与最优模型的测试集均方误差。你可以比较自由地设计超参数搜索策略，感受调参的过程。

   （不强制指定搜索超参数组合数量，如果你是个经验丰富的开发者，你完全可以仅通过少量调整确定最优参数组合。模型效果不影响给分，但请注意，如果你搜索的超参数组合数 <6 **并且**模型表现较差，会扣除少量分数。）

2.6 随机梯度下降 (9pt)

当训练数据集非常大时，梯度下降算法需要遍历整个数据集，因此效率较低。实际中用的往往是**随机梯度下降**（SGD）算法。令 $f_i(\theta) = (h_\theta(x_i) - y_i)^2$ 为第 i 个数据点上的平方误差，则

$$J(\theta) = \frac{1}{m}\sum_{i=1}^{m} f_i(\theta) + \lambda\theta^T\theta$$

在 SGD 中，每一步会随机采样一个小批量（batch）$\{(x_{i_k}, y_{i_k})\}_{k=1}^{n}$ 来计算目标函数

$$J_{\mathrm{SGD}}(\theta) = \frac{1}{n}\sum_{k=1}^{n} f_{i_k}(\theta) + \lambda\theta^T\theta$$

和梯度方向，其中 $n \ll m$ 并且 i_k 从 $\{1, 2, \ldots, m\}$ 中独立同分布采样。

1. 请直接写出 $J_{\mathrm{SGD}}(\theta)$ 对应的梯度 $\nabla J_{\mathrm{SGD}}(\theta)$ 的表达式。
2. 补全函数 `stochastic_grad_descent`，实现**随机梯度下降**算法。
3. 随机梯度下降算法的噪音较大，因此模型较难收敛。你需要选择合适的批大小：固定 $\lambda = 0$，并根据 2.5 的结果固定选取合适的超参，从批大小 1 开始，尝试各种不同的批大小，并记录随着批大小逐渐增大时，训练曲线发生的变化。注意：在随机梯度下降中，小批量的训练损失函数噪声较大，难以清晰反映模型收敛情况，所以要通过验证集上的全批量损失来判断模型收敛情况。补充 main 函数，完成必要的训练和记录过程（使用不同的超参数调用 `stochastic_grad_descent` 函数，记录实验结果）

2.7 广义线性模型：Softmax 回归 (5pt)

线性模型也可用于处理（多）分类任务。假设类别数为 K，输入数据为 $\mathbf{x} \in \mathbb{R}^n$，权重参数为 $\mathbf{W} \in \mathbb{R}^{K\times n}$，偏置项为 $\mathbf{b} \in \mathbb{R}^K$，模型的输出为：

$$\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$$

可使用 Softmax 函数将模型输出 $\mathbf{z}$ 转换为类别的概率分布：

$$\hat{\mathbf{y}} = [\hat{y}_1, \hat{y}_2, \cdots, \hat{y}_K]^T, \quad \hat{y}_i = \frac{\exp z_i}{\sum_{j=1}^{K}\exp z_j}$$

其中 $\hat{y}_i$ 表示输入 $\mathbf{x}$ 属于第 i 类的预测概率。

将数据的真实类别表示为独热 (one-hot) 向量 $\mathbf{y} = [y_1, y_2, \cdots, y_K]^T$ 的形式，其中 $y_k = 1, \forall i \neq k, y_i = 0$（k 是数据的真实类别）。可以得出 Softmax 函数的负对数似然损失函数（交叉熵损失函数）：

$$\mathcal{L} = -\log \hat{y}_k = -\mathbf{y}^T \log \hat{\mathbf{y}}$$

相信同学们完全有能力仿照 2.3 的过程，推导出 $\mathcal{L}$ 关于 $\mathbf{z}$ 的梯度和 $\mathcal{L}$ 关于 $\mathbf{W}$ 和 $\mathbf{b}$ 的梯度，再实现梯度下降算法。但此题中，我们不再重复练习 numpy 的使用，助教已经使用 pytroch 仓库实现了 softmax 线性分类器的训练和测试。请同学们借助 pytorch 官方文档或大模型读懂代码，完成以下任务：

1. 代码中是如何定义损失函数的，找到对应的调包实现（你应该会看到助教通过一个 torch.nn 的包实现了损失函数)，并查阅文档回答，这个损失函数的计算和我们推导出的负对数似然损失函数一致吗?
2. 运行并汇报测试结果（结果不理想是正常现象）。关于此结果的分析，将留到 2.9 进行。

2.8 广义线性模型：Poisson 分布 (6pt)

现考虑 y 服从 Poisson 分布的情况，即

$$\mathbf{y} \sim \mathbf{Poisson}(\lambda)$$

我们知道 Poisson 分布的均值和方差均为 λ，我们使用线性模型作为预测器，对数函数作为 link function：

$$\eta = \mathbf{w}^T\mathbf{x}$$

$$\lambda = \mathbb{E}[y] = \mathbf{exp}(\eta) = \mathbf{exp}(\mathbf{w}^T\mathbf{x})$$

根据广义线性模型的通用形式

$$\mathbf{P}(\mathbf{y};\theta,\phi) = \exp\left(\frac{\mathbf{y}\theta - \mathbf{b}(\theta)}{\phi} + \mathbf{c}(\mathbf{y},\phi)\right)$$

完成以下任务：

1. 推导对应的负对数似然损失函数（你可以根据 Poisson 分布的具体形式确定 $\mathbf{c}(\mathbf{y},\phi)$)。
2. 直接写出最终的优化目标（课件中的结果使用了对 $\mathbf{y}!$ 的近似。此处也可以将 y 视为常数，故你的答案可以跟课件不完全一致，只要 $\mathbf{w}$ 相关项一致即为正确)。

2.9 扩展题目：数据生成、K 近邻和 Softmax 实验结果的分析（10pt）

本题为开放性题目，助教仅提供研究的问题，不提供具体执行要求或预期结果。请大家根据自己的兴趣开展实验并分析实验结果。你并不需要也不建议覆盖多个话题，如果你的分析比较完整，只讨论一个话题也可以拿满 10 分；浅尝辄止地覆盖所有话题也不会获得高分。

1. 话题 1：`generate_data.py` 文件是助教生成本次作业使用数据的文件。你可以阅读此代码，看懂助教生成数据的方式，思考每一步操作的合理性。指出你认为可以改进的地方，或者联系你做作业的过程，看看有什么新发现。
2. 话题 2：在 2.2.2 中，你最后应该发现 $i = 1, 2, 3$ 时，$x_i, cos(x_i), x_i^2$ 这三项中都只有一项与 y_1 有关。实际上，助教尝试过让 y_1 同时依赖 x_3 和 x_3^2，但是却发现模型最后学到的结果（尤其在正则化系数较大时）虽然能够较好地拟合，但并不符合真实函数。你可以尝试复现助教的实验现象，并尝试解释原因。
3. 话题 3：尝试使用 K 近邻算法拟合 y_2 或 y_3。你**可以**调包实现 K 近邻算法，调整距离计算方式，K 值等。你也可以用助教的代码生成更多的数据，看看数据量的多少会对 K 近邻产生什么影响。
4. 话题 4：2.7.3 中，你应该会看到模型最后的准确率并不特别理想。尝试分析其原因（比如数据中可能存在过多噪声等)。根据助教生成数据的代码，你可以知道“真正最优的函数”，由此计算贝叶斯错误率，看看能有什么发现。

3 支持向量机 (40pt)

在本题，我们将使用支持向量机（Support Vector Machine）对文本数据进行情绪检测。

3.1 次梯度 (3pt)

对于 $f: \mathbf{R}^d \to \mathbf{R}$，在 x 如果对于所有 z，

$$f(z) \geq f(x) + g^T(z - x),$$

则 $g \in \mathbf{R}^d$ 为 x 处的**次梯度**（subgradients）。次梯度并不唯一，f 在 x 处的次梯度集合记为 $\partial f(x)$，那么 $g \in \partial f(x)$。大部分时候，例如在进行次梯度下降的时候，我们只需要次梯度集合中的一个次梯度即可，这时也可以写作 $\partial f(x) = g$。

基于以上的定义，你需要给出合页损失函数（Hinge Loss）的次梯度。合页损失如下。

$$J(w) = \max\left\{0, 1 - yw^T x\right\}$$

3.2 硬间隔支持向量机 (6pt)

给定线性可分的数据集 $(x_1, y_1), \ldots, (x_m, y_m) \in \mathbf{R}^d \times \{-1, 1\}$，支持向量机模型希望能找到一个超平面将两个类别完全区分开，即找到 $w \in \mathbf{R}^d, b \in \mathbf{R}$ 使得

$$y_i(w^T x_i + b) > 0, \quad 1 \leq i \leq m$$

即所有标签 $y = 1$ 的 x 都在超平面 $\left\{x \mid w^T x + b = 0\right\}$ 的一侧，并且所有标签 $y = -1$ 的 x 都在另一侧。同时，支持向量机希望数据关于该超平面的间隔尽量大。不失一般性，我们可以设距离该超平面最近的点在 $w \cdot x + b = \pm 1$ 上，此时问题可以转化为带约束优化问题：

$$\begin{aligned} \min_{w \in \mathbf{R}^d, b \in \mathbf{R}} \quad & \frac{1}{2}\|w\|_2^2 \\ \text{s.t.} \quad & y_i(w^T x_i + b) \geq 1, \quad 1 \leq i \leq m \end{aligned} \tag{1}$$

设拉格朗日变量 $\mu_i \geq 0, \quad 0 \leq i \leq m$ 对应上述问题的 m 个约束，则问题 (1) 对应的拉格朗日函数为：

$$L(w, b, \mu) = \frac{1}{2}\|w\|_2^2 - \sum_{i=1}^{m} \mu_i [y_i(w^T x_i + b) - 1] \tag{2}$$

1. 请根据上述内容，写出问题 (1) 的 KKT 条件。
2. 证明满足 KKT 条件的解 w 一定是训练数据 $x_1, x_2, ..., x_m$ 的线性组合。设解 $w = \sum_{i=0}^{m} \alpha_i x_i$，若 $\alpha_i \neq 0$，对应的 x_i 称为支持向量。请说明支持向量均位于分隔平面 $w \cdot x + b = \pm 1$ 上。

3.3 软间隔支持向量机 (9pt)

线性可分是一种理想的情况，现实数据集经常会有噪声，无法约束所有的数据点都能被正确分类，因此实际中的支持向量机都会引入软间隔，目标是让不满足约束的数据点尽可能少，即

$$\begin{aligned} \min_{w\in\mathbf{R}^d, b\in\mathbf{R}, \xi\in\mathbf{R}^m} \quad & \frac{\lambda}{2}\|w\|_2^2 + \frac{1}{m}\sum_{i=1}^{m}\xi_i^p. \\ \text{s.t.} \quad & y_i(w^T x_i + b) \geq 1 - \xi_i \\ & \xi_i \geq 0, \quad 1 \leq i \leq m \end{aligned} \tag{3}$$

其中，系数 $p \geq 1$ 用于控制对数据点不满足约束的惩罚力度。

1. 求 $p \geq 1$ 时，该问题的拉格朗日（Lagrange）方程。
2. 求 $p = 1$ 时，该问题的对偶形式（Dual Form）。
3. 在求出对偶问题后，使用 Sequential Minimal Optimization (SMO) 算法，可以实现软间隔支持向量机的实际求解。$p = 1$ 时，上文中的优化问题也可以等价表示为：

$$\min_{w\in\mathbf{R}^d, b\in\mathbf{R}} \frac{\lambda}{2}\|w\|^2 + \frac{1}{m}\sum_{i=1}^{m}\max\left\{0, 1 - y_i(w^T x_i + b)\right\}. \tag{4}$$

此时，SVM 求解还有另外一种基于**次梯度下降**的算法。证明此时软间隔支持向量机中 $J_i(w) = \frac{\lambda}{2}\|w\|^2 + \max\left\{0, 1 - y_i(w^T x_i + b)\right\}$ 的次梯度由下式[1]给出（提示：参考合页损失函数的次梯度）。

$$\partial J_i|_w = \begin{cases} \lambda w - y_i x_i & \text{for } y_i(w^T x_i + b) < 1 \\ \lambda w & \text{for } y_i(w^T x_i + b) \geq 1. \end{cases}, \quad \partial J_i|_b = \begin{cases} -y_i & \text{for } y_i(w^T x_i + b) < 1 \\ 0 & \text{for } y_i(w^T x_i + b) \geq 1. \end{cases}$$

3.4 核方法 (9pt)

非线性分类问题是指通过利用非线性模型才能很好地进行分类的问题，面对这类问题，我们可以使用**核技巧**（kernel trick）。核技巧将数据 $\boldsymbol{x} = (x_1, x_2, ..., x_d)^T \in \mathcal{X}$ 通过基函数 $\boldsymbol{\Phi}(\boldsymbol{x}) = (\phi_1(\boldsymbol{x}), \phi_2(\boldsymbol{x}), ..., \phi_n(\boldsymbol{x}))^T$ 映射到高维空间，并定义基函数导出的核函数为 $k(\boldsymbol{x}_1, \boldsymbol{x}_2) = \boldsymbol{\Phi}(\boldsymbol{x}_1) \cdot \boldsymbol{\Phi}(\boldsymbol{x}_2)$。通过对问题的变形，可以用核函数替代高维空间上的点积操作，从而简化问题的计算。

要验证任意的函数 $k(\boldsymbol{x}_1, \boldsymbol{x}_2)$ 是否是核函数，除了直接找出其对应的基函数 $\boldsymbol{\Phi}(\boldsymbol{x})$，还可以通过核矩阵是否对称半正定进行判断。Mercer 条件指出，对于紧集 $\mathcal{X} \subset \mathbf{R}^d$ 和对称函数 $k : \mathcal{X} \times \mathcal{X} \to \mathbf{R}$，$k$ 是核函数当且仅当对于任意 $(\boldsymbol{x}_1, \boldsymbol{x}_2, ..., \boldsymbol{x}_m) \subseteq \mathcal{X}$，矩阵 $\boldsymbol{K} = [k(\boldsymbol{x}_i, \boldsymbol{x}_j)]_{i,j} \in \mathbf{R}^{m\times m}$ 对称半正定，即矩阵 $\boldsymbol{K}$ 对称且对于任意列向量 $\boldsymbol{a} = (a_1, a_2, ..., a_m)^T \in \mathbf{R}^{m\times 1}$，

$$\boldsymbol{a}^T \boldsymbol{K} \boldsymbol{a} = \sum_{i,j=1}^{m} a_i a_j k(\boldsymbol{x}_i, \boldsymbol{x}_j) \geq 0$$

1. 对于定义在 $\mathbf{R}^n \times \mathbf{R}^n$ 上的对称函数 $k(\boldsymbol{x}, \boldsymbol{x}') = cos\angle(\boldsymbol{x}, \boldsymbol{x}')$，证明矩阵 $\boldsymbol{K} = [k(\boldsymbol{x}_i, \boldsymbol{x}_j)]_{i,j} \in \mathbf{R}^{m\times m}$ 对称半正定，并求出其对应的基函数 $\boldsymbol{\Phi}(\boldsymbol{x})$ 使 $k(\boldsymbol{x}_1, \boldsymbol{x}_2) = \boldsymbol{\Phi}(\boldsymbol{x}_1) \cdot \boldsymbol{\Phi}(\boldsymbol{x}_2)$。（$\angle(\boldsymbol{x}, \boldsymbol{y})$ 表示向量 $\boldsymbol{x}$，$\boldsymbol{y}$ 的夹角）

[1] $\partial J_i|_w$ 表示次梯度 ∂J_i 在 w 对应的维度上的分量，$\partial J_i|_b$ 同理。

2. 证明对任意两个核函数 $k_1, k_2 : \mathcal{X} \times \mathcal{X} \to \mathbf{R}$，它们的和 $k_s(\boldsymbol{x}, \boldsymbol{x}') = k_1(\boldsymbol{x}, \boldsymbol{x}') + k_2(\boldsymbol{x}, \boldsymbol{x}')$ 仍是核函数，乘积 $k_p(\boldsymbol{x}, \boldsymbol{x}') = k_1(\boldsymbol{x}, \boldsymbol{x}') \cdot k_2(\boldsymbol{x}, \boldsymbol{x}')$ 也是核函数。（提示：对任意对称半正定矩阵 $\boldsymbol{K}$，存在矩阵 $\boldsymbol{M}$ 使得 $\boldsymbol{K} = \boldsymbol{M}\boldsymbol{M}^T$）
3. 已知非线性映射 $\boldsymbol{\Phi}$ 和其对应的核函数 $k(\cdot, \cdot)$，若在 3.3 节中的支持向量机 (取 p=1) 中使用该核函数，请写出对偶问题形式，并指出如何用使用核函数计算测试样本的分类。

3.5 情绪检测 (13pt)

我们将使用**线性软间隔支持向量机**进行情绪检测，类别包括：开心（joy）和伤心（sadness）。我们在 `start_code.py` 中已经实现了数据集的加载与预处理，你也可以自己实现。你需要完成

1. 在函数 `linear_svm_subgrad_descent` 中实现 SVM 的随机次梯度下降算法，并在情绪检测数据集上进行训练（提示：可参考 2.6 节随机梯度下降的代码）。
2. 在函数 `kernel_svm_subgrad_descent` 中实现基于核函数的非线性 SVM（例如基于线性核或者高斯核），调整相关的超参数，记录它在测试集上的准确率。核函数的引入能提高当前模型的准确吗？试解释原因。
3. 计算并汇报最终的 SVM 模型在验证集上的准确率，F1-Score 以及混淆矩阵（Confusion Matrix）。